

Clase 18

# Testing

---

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra  
aaossa@ing.puc.cl

1. Recuerden la Entrega 1
2. Sala de ayuda E1 mañana

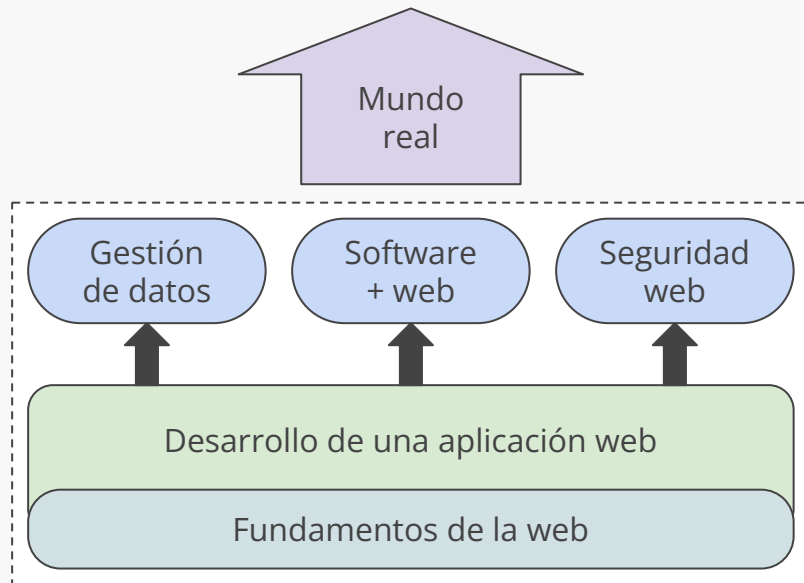
# Anuncios

---

# Contenidos del curso

---

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real



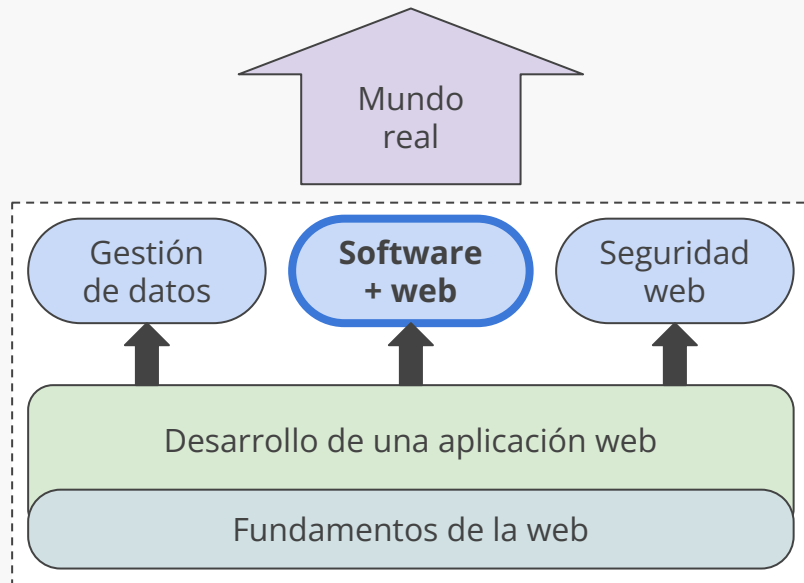
# Contenidos del curso

---

## Software + web

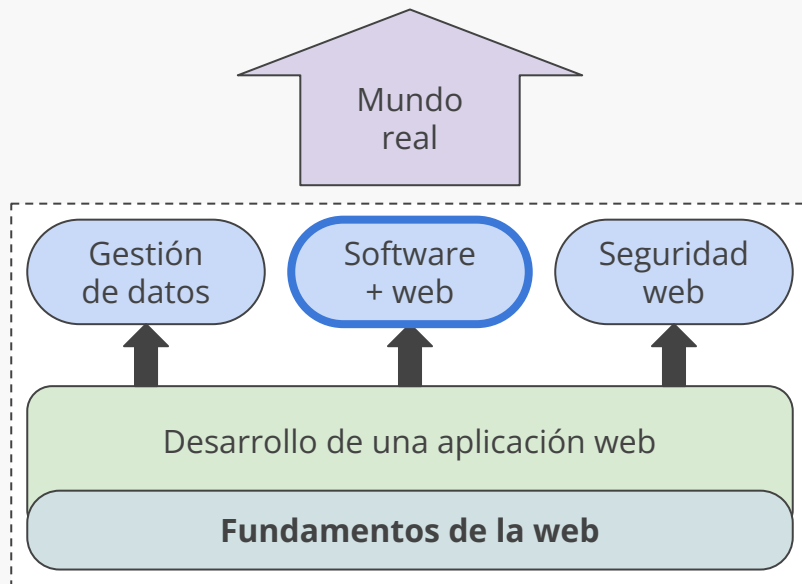
Hablaremos sobre elementos que apuntan a expandir y **mejorar la experiencia de desarrollo web** tales como *testing* y *websockets*...

... para reconocer e identificar tecnologías de vanguardia y que permiten **buenas prácticas en desarrollo web**



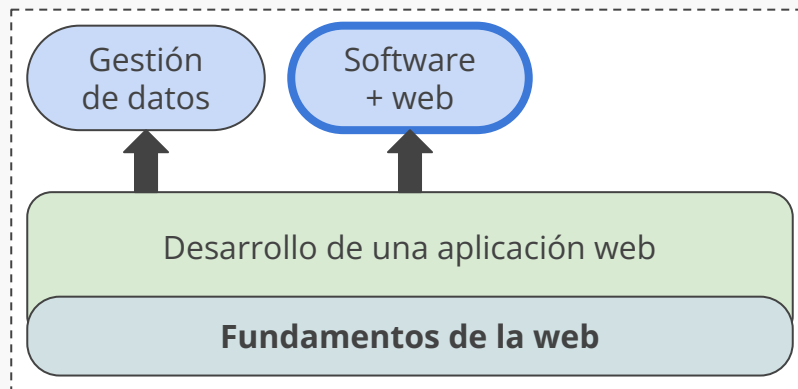
# Contenidos del curso

---



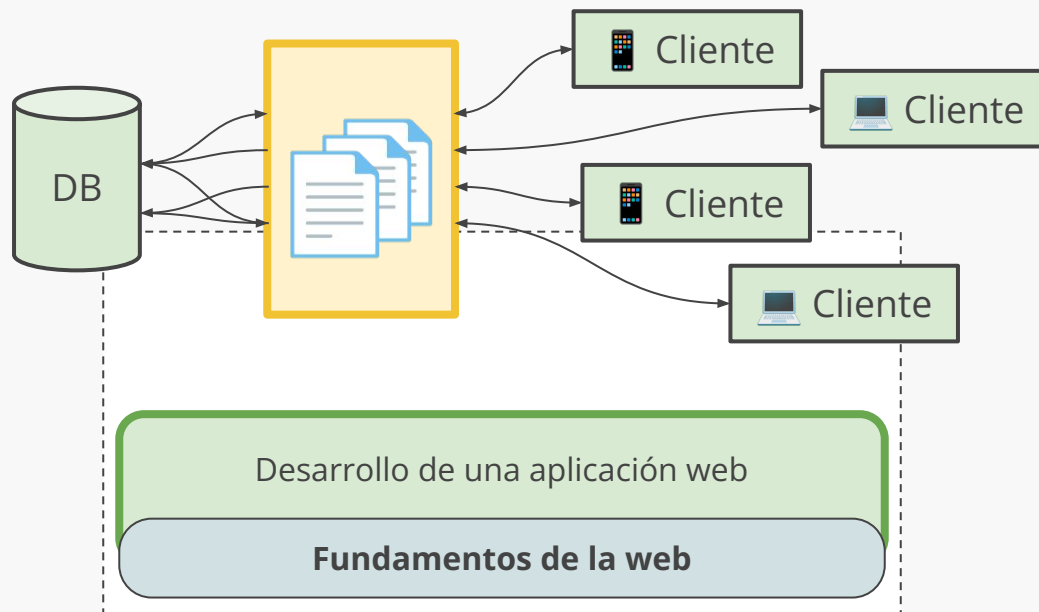
# Contenidos del curso

---



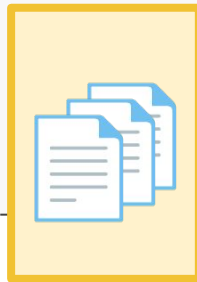
# Motivación

---



# Motivación

---



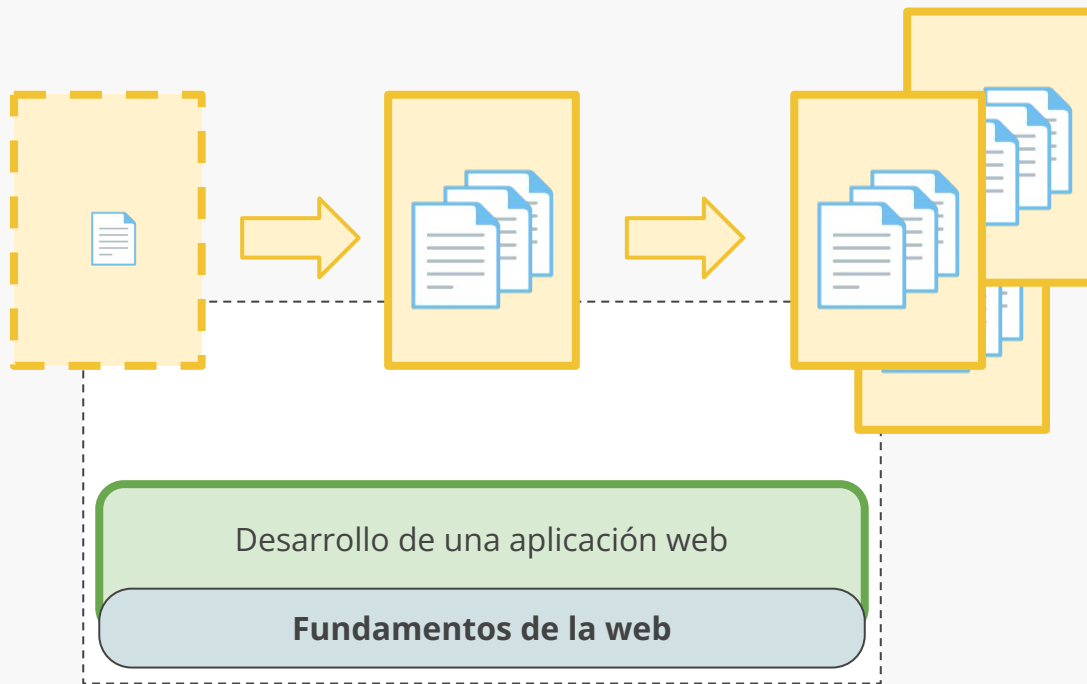
Desarrollo de una aplicación web

**Fundamentos de la web**



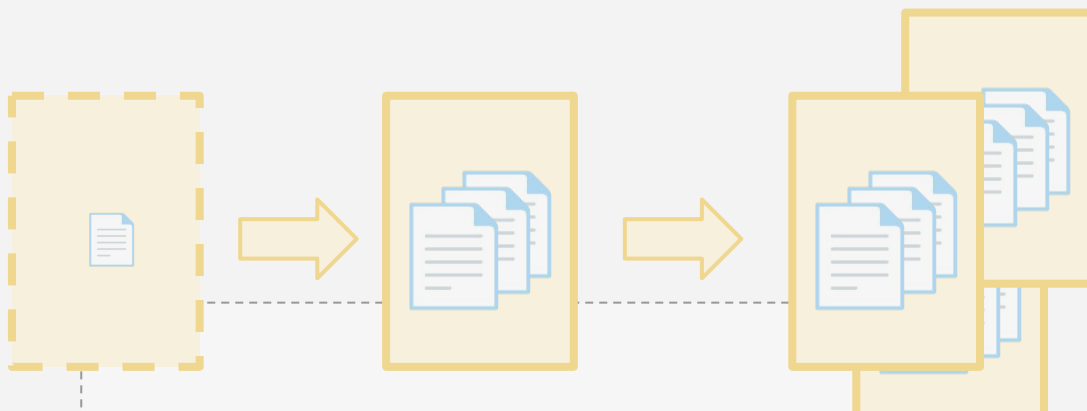
# Motivación

---



# Motivación

---

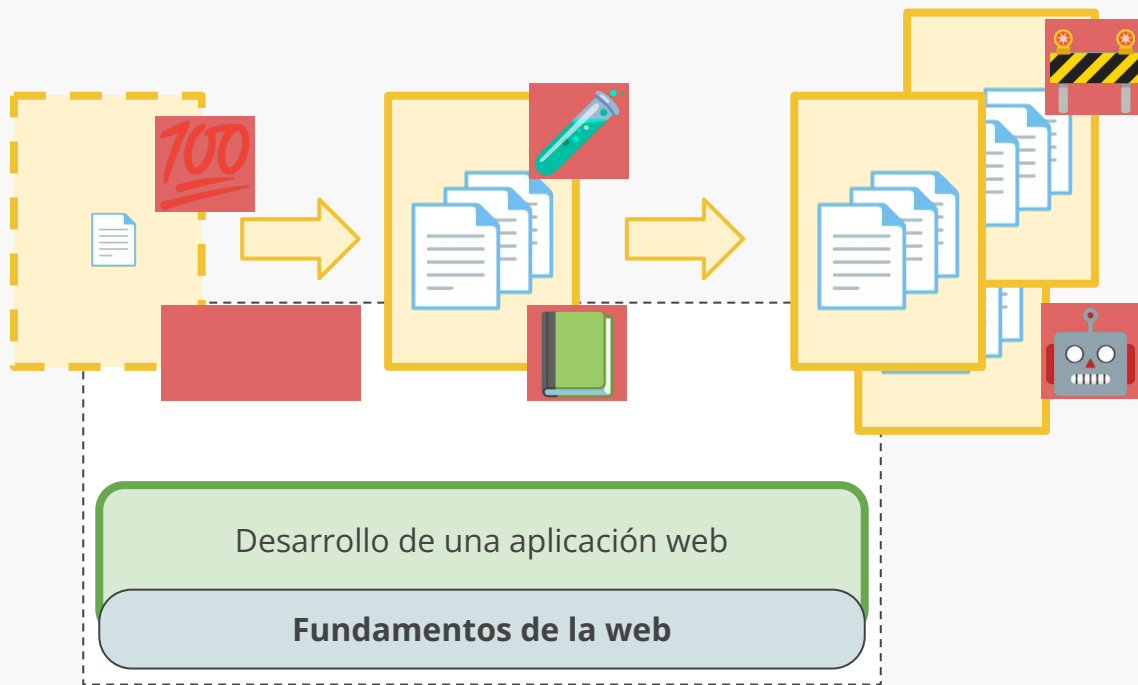


**Pregunta#¿Cómo desarrollar “bien”?**

Fundamentos de la web

# Motivación

---



# Aseguramiento de Calidad (*Quality Assurance, QA*)

**Pregunta#** ¿Existe el software sin errores?

**No, el software no es infalible**

Toda pieza de software es hecha por humanos,  
y los humanos cometemos errores

# ¿Qué podemos hacer?

---

Lo que queremos lograr es: minimizar errores en el software que construimos, y verificar que el software funcione y sea mantenible. Para lograrlo, contamos con estas técnicas:

*Testing*



*Análisis estático*



*Code review*



# *Testing*

---



Diseño de pruebas automatizadas con el objetivo de someter al software a diferentes escenarios

# Testing

---

Probar que la funcionalidad cumple lo especificado y también que es robusta frente a potenciales errores. Bastante común en librerías de código abierto, en piezas de software usadas en producción... aunque poco común en proyectos personales

(Más sobre esto la **futura clase de Testing** y en **IIC3745 - Testing**)

## Clase 18#*Testing*

(Más sobre esto la **futura clase de *Testing*** y en **IIC3745 - Testing**)



**Menti***#Hablemos de testing*

# ¿Cómo van con la Entrega 1?



# Sobre la clase anterior

Clase 17 - Tecnologías adicionales:

TypeScript

WebAssembly

WebSockets

Progressive Web Apps (PWA)

# Menciona algo que hayas aprendido en esta clase

Que se pueden compilar  
otros lenguajes a wasm

guardar tokens para apis en  
cookies es suficientemente  
bueno; ojalá no en el local  
storage

Typescript permite  
controlar mejor los  
errores

Existe Rooms en  
websockets para hacer  
lobbys/ grupos de gente  
conectada

Es terrible escribir  
directamente en web  
assembly

eeemm

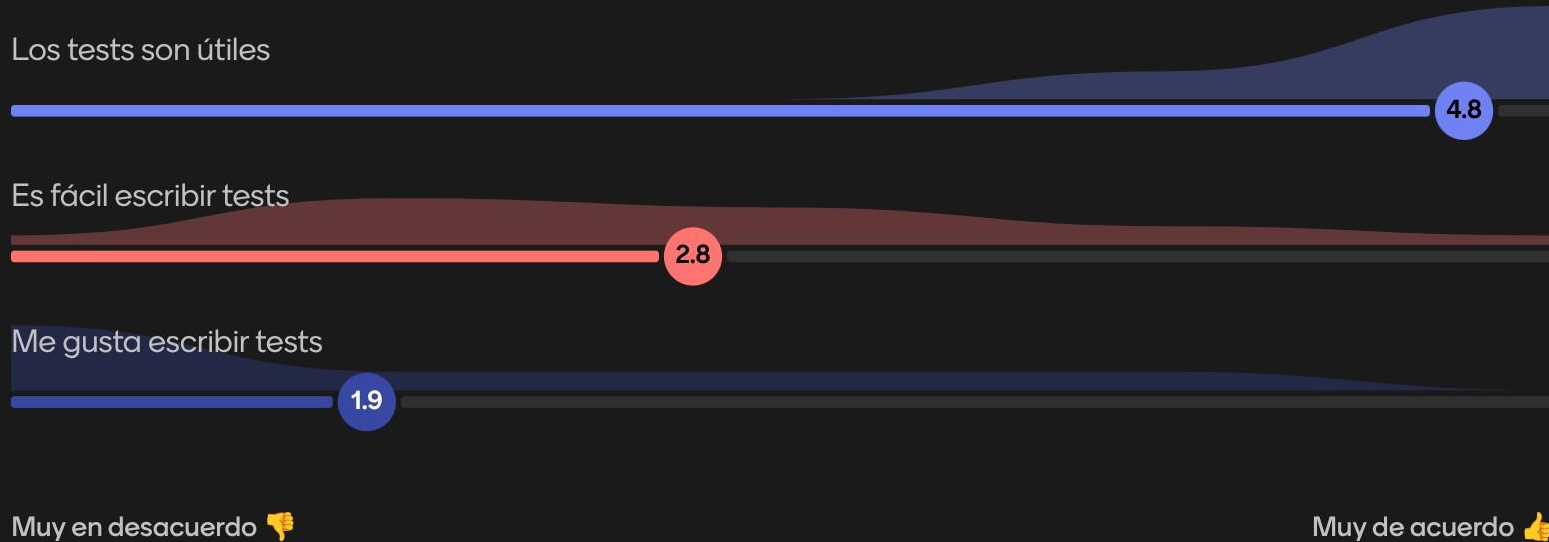
# Sobre la clase de hoy

Clase 18 - Testing

# ¿Has escrito algún test? 🙄



# ¿Qué tan de acuerdo estás con esto? 📄



# ¿Qué podríamos testear en una app como la del proyecto? 🙄

que el no rellenar  
campos obligatorios sea  
avisado

crear un usuario y que falle  
si ya está usado el  
username, o que falle si la  
clave es muy corta

Que las request retornen  
lo esperado

que sea facil de usar

UX



# ¿Qué cosas creen que NO son testeables? 🤔

nada xd

Los test

que sea intuitiva de usar

Calidad del código?

UI

Comportamiento  
responsivo

Si es responsiva la  
aplicacion

ux

# ¡Gracias!

# Agenda

---

## 🌟 Testing

Tests de *back-end*

Tests de *front-end*

# ¿Qué es *testing*?

---

*Testing* se refiere al proceso de ejecutar un programa con el objetivo de encontrar un error. Por esta razón, un **buen test** es uno con una alta probabilidad de encontrar un error. Luego, un **test exitoso** es el que descubre un error que no se conocía

## **Beneficios:**

- Verificar que la funcionalidad cumple con lo especificado
- Verificar que la implementación es robusta frente a potenciales errores
- Base sólida para construir sobre lo implementado
- Sirve como documentación

# ¿Qué es *testing*?

---

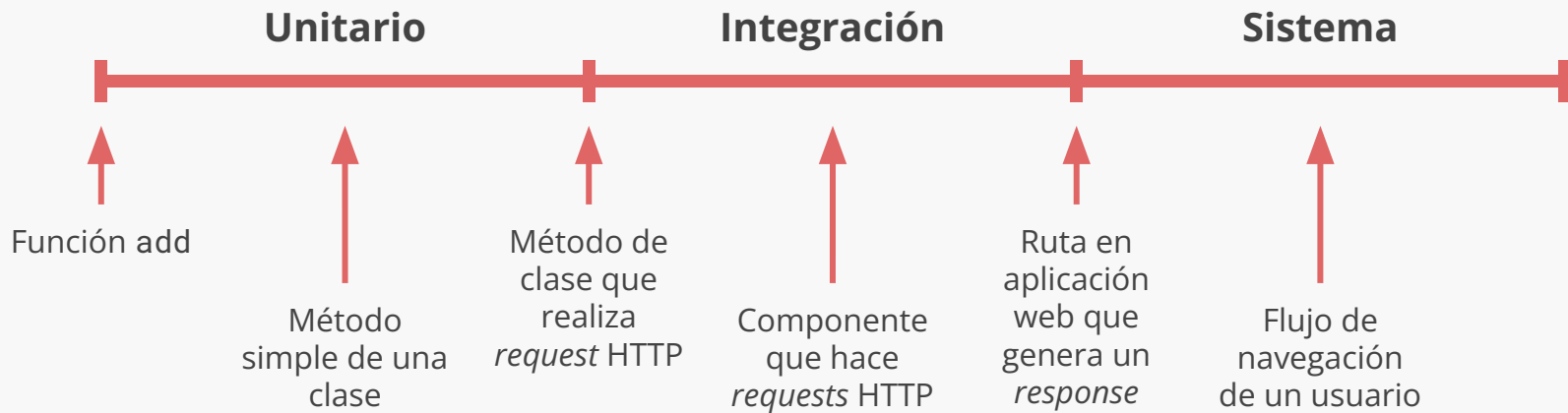
Pruebas que permiten **verificar** y **validar** el funcionamiento del *software*:

- **Verificación:** Construir correctamente el software. ¿Estamos construyendo el producto correctamente? Es el proceso de evaluar y confirmar que el producto cumple con las especificaciones y requisitos
- **Validación:** Relevancia y utilidad del software. ¿Hemos construido el producto correcto? Es el proceso de confirmar que el producto satisface las necesidades y expectativas reales de los usuarios finales

**Primero ocurre la etapa de verificación,  
y al final la etapa de validación**

# ¿Qué es *testing*?

---



# ¿Qué es *testing*?

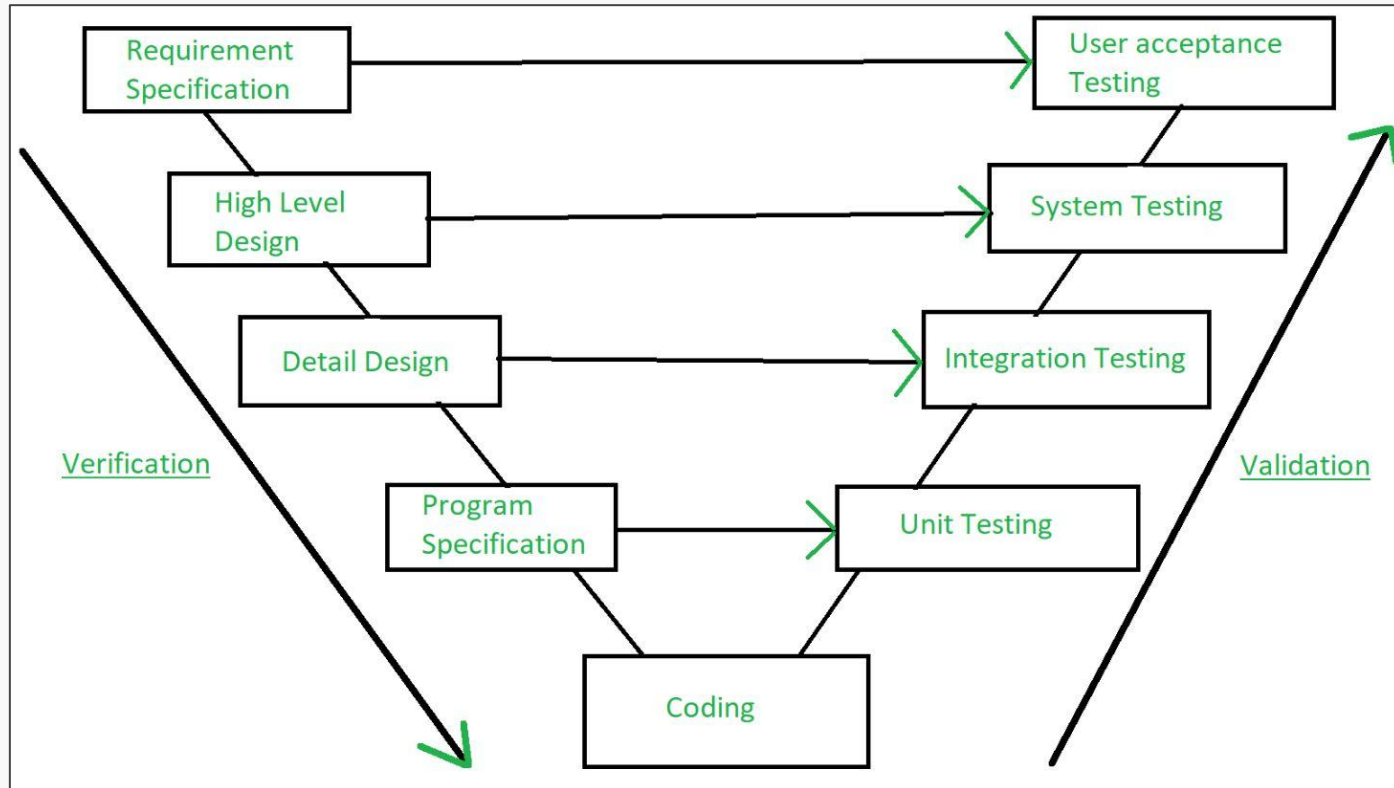


Figura: [Verification and Validation in Software Engineering | GeeksforGeeks](#)

# *Unit testing* (Tests unitarios)

---

- Se prueban componentes/unidades individuales de un *software*
- Una unidad puede ser una función, procedimiento, módulo u objeto
- El objetivo es validar que cada unidad haga lo que se espera
- Se pueden realizar antes, durante o después de programar
- Se debe tener un control de los resultados esperados (inputs y outputs)



# Unit testing: un ejemplo

---

Supongamos que tenemos una función para validar un teléfono

Podemos testear dos cosas: que retorne *true* cuando el teléfono sea válido y que retorne *false* cuando no lo sea

```
function validatePhoneNumber(input) {  
  if (typeof input !== 'string') return false;  
  const match = input.match(/^(\+1)?\d{10}$/);  
  return Boolean(match);  
}  
  
module.exports = { validatePhoneNumber };
```

¿Cuáles casos podríamos usar?

# Unit testing: un ejemplo

---

Supongamos que tenemos una función para validar un número de teléfono.

Podemos testear que la función retorne *true* cuando el número es válido y que retorne *false* cuando no lo sea.

¿Cuáles casos podríamos testear?

```
const { validatePhoneNumber } = require('./phoneValidator');

describe('validatePhoneNumber()', () => {
  test('returns true for valid phone numbers', () => {
    // simple 10 digits
    expect(validatePhoneNumber('1234567890')).toBe(true);
    // with country code and separators
    expect(validatePhoneNumber('+1 (123) 456-7890')).toBe(true);
  });

  test('returns false for invalid phone numbers', () => {
    // too few digits
    expect(validatePhoneNumber('123-45-6789')).toBe(false);
    // wrong type
    expect(validatePhoneNumber(1234567890)).toBe(false);
  });
});
```

# Unit testing: mocks y stubs

---

**Stubs:** Son imitaciones de objetos, que **proveen resultados predefinidos** para ciertas invocaciones sobre ellos. Útiles para probar componentes de forma aislada

**Mocks:** Son imitaciones de objetos, de las cuales **se espera que ciertos métodos sean invocados** durante un *test*. Si no es así, el *test* debería fallar

En resumen, si un test solo necesita proveer datos de entrada, se usa un *stub*; si además se necesita verificar que la interacción ocurra de una forma esperada, se usa un *mock*

# Unit testing: otro ejemplo

---

¿Qué debería hacer un test para este método? Sería conveniente verificar que si la tarjeta es rechazada, se retorne un mensaje apropiado

Como vemos, la clase `AccountServices` crea una instancia de `BankWebService` para consultar, pero para poder testear correctamente deberíamos entregarle una instancia propia. Invertamos la dependencia: en lugar de que la clase cree el *webservice*, le entregamos uno: entreguemos un *mock*

```
public class AccountServices
{
    private IBankWebService _webService = new BankWebService();

    public string RegisterUser(string username, string creditCard)
    {
        AddUserToDatabase(username);

        bool success = _webService.BillUser(creditCard);

        if (success == false)
            return "Your credit card was declined"
        else
            return "Success!"
    }
}
```

# Unit testing: otro ejemplo

---

Ahora, como la clase que llama el método recibe cualquier *webservice*, podremos crear nuestro propio servicio falso, un *mock*, para poder testear tanto si el resultado es aceptado como rechazado

```
public class AccountServices
{
    public AccountServices(IBankWebService webService)
    {
        _webService = webService;
    }

    private IBankWebService _webService;

    public string RegisterUser(string username, string creditCard)
    {
        AddUserToDatabase(username);

        bool success = _webService.BillUser(creditCard);

        if (success == false)
            return "Your credit card was declined"
        else
            return "Success!"
    }
}
```

# Unit testing: otro ejemplo

---

Al usar este objeto falso, cuando se quiera verificar la tarjeta de un usuario, nuestro objeto falso siempre retorna "falso"

Nuestro test unitario ahora verifica que si la llamada al banco falla, RegisterUser() va a retornar el mensaje correcto

```
public class FakeBankWebService : IBankWebService
{
    public bool BillUser(string creditCard)
    {
        return false; // our fake object always says billing failed
    }
}

public void TestUserIsRemoved()
{
    IBankWebService fakeBank = FakeBankWebService();

    AccountServices services = new AccountServices(fakeBank);

    string registrationResult = services.RegisterUser("test_username");

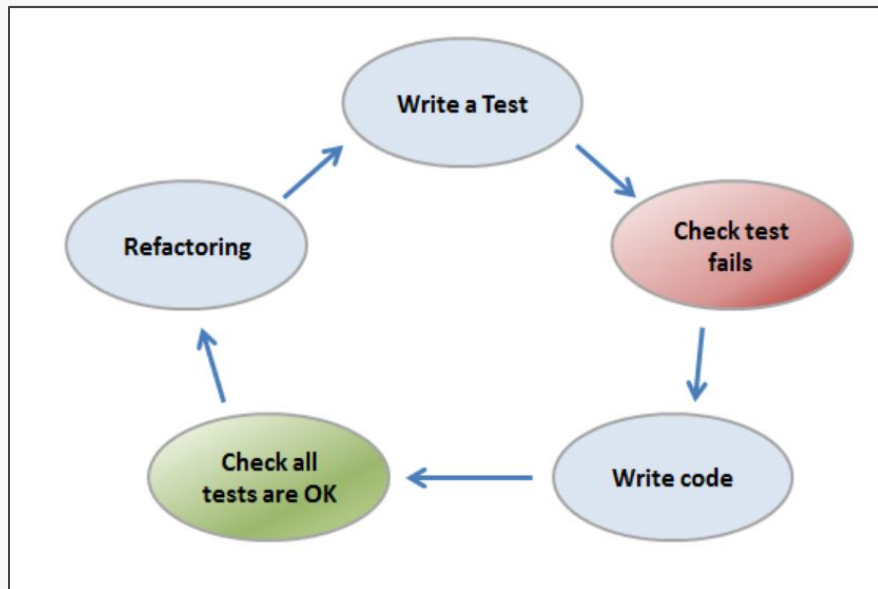
    Assert.AreEqual("Your credit card was declined", registrationResult);
}
```

# Unit testing: Test Driven Development

---

Metodología basada en desarrollar código en ciclos pequeños e iterativos que incluyen:

1. Diseñar *tests* para un requerimiento
2. Desarrollar código hasta pasar *tests*
3. Mejorar la implementación (*refactor*)



# *Integration testing* (Tests de integración)

---

- Se prueban módulos del *software* como conjunto
- Su objetivo es exponer defectos en la integración entre módulos
- Si bien cada componente debe pasar sus pruebas, es importante revisar si estas interactúan como se espera



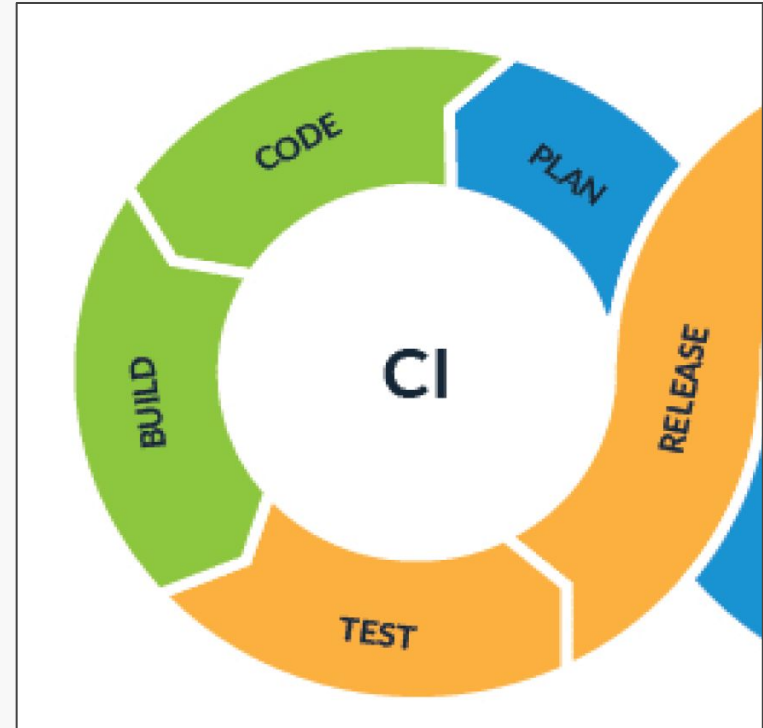
# Continuous Integration (CI)

Integración Continua es una metodología que fomenta la integración de cambios en el código de manera continua, durante el desarrollo

Busca disminuir los costos de integración y detectar fallas de forma temprana

Es buena práctica automatizar este proceso con herramientas como:

- Jenkins CI
- Circle CI
- Travis CI
- (CloudBees) Codeship CI



# *System testing* (Tests de sistema)

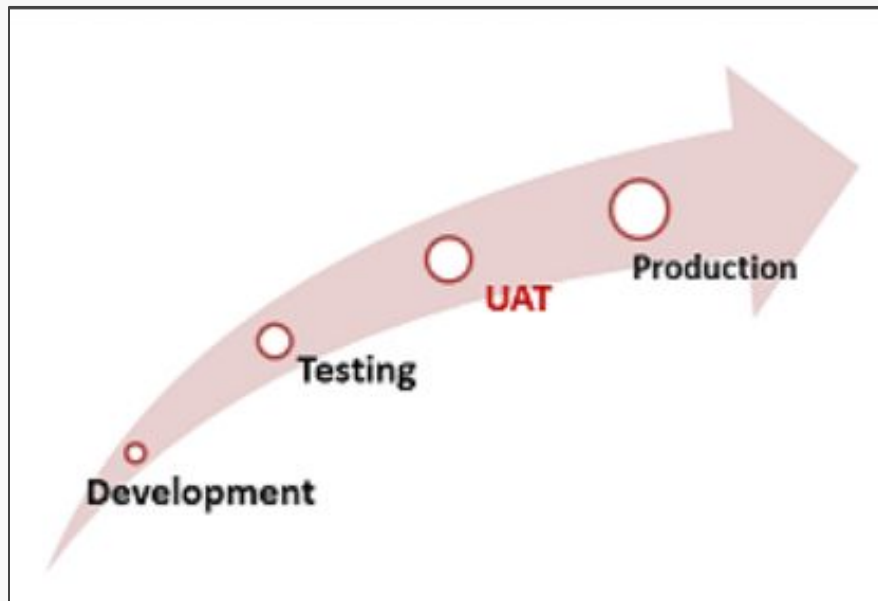
---

- Se valida la completa integración del producto de software
- Su propósito es evaluar las especificaciones de manera *end-to-end*
- Usualmente, se conecta con otras piezas de *software/hardware*
- Se realizan una serie de diferentes pruebas para probar el sistema completo
- Realizado antes de pasar el producto a producción

# Acceptance testing (Tests de aceptación)

---

- Pruebas realizadas por *end users* o clientes para verificar/aceptar el software antes de que la aplicación se pase a producción
- Solo se realiza en la fase final de *testing*, cuando todos los *tests* unitarios, de integración y de sistema ya fueron completados (y pasados)



# Agenda

---

Testing

✨ **Tests de *back-end***

Tests de *front-end*

# Estructura de un test

---

**Dado** un conjunto de condiciones iniciales

**Cuando** se ejecutan ciertas acciones

**Entonces** se cumplen las expectativas

# Estructura de un test

---

- Dado** un conjunto de condiciones iniciales ▶ **Dado** que hoy hace frío
- Cuando** se ejecutan ciertas acciones ▶ **Cuando** salgo con poco abrigo
- Entonces** se cumplen las expectativas ▶ **Entonces** me da frío

# Estructura de un test

---

**Dado** que hoy hace frío

**Cuando** salgo con poco abrigo

**Entonces** me da frío

```
describe('Dado que hoy hace frío', () => {  
  describe('Cuando salgo con poco abrigo', () => {  
    test('Entonces me da frío', () => {  
      expect(body).toBe('cold');  
    });  
  });  
});
```

# Jest · Delightful JavaScript Testing: Framework de testing para JavaScript

Support Ukraine  [Help Provide Humanitarian Aid to Ukraine.](#)

JEST

29.7 ▼

[Docs](#)

[API](#)

[Help](#)

[Blog](#)

 [English](#) ▼



 Search

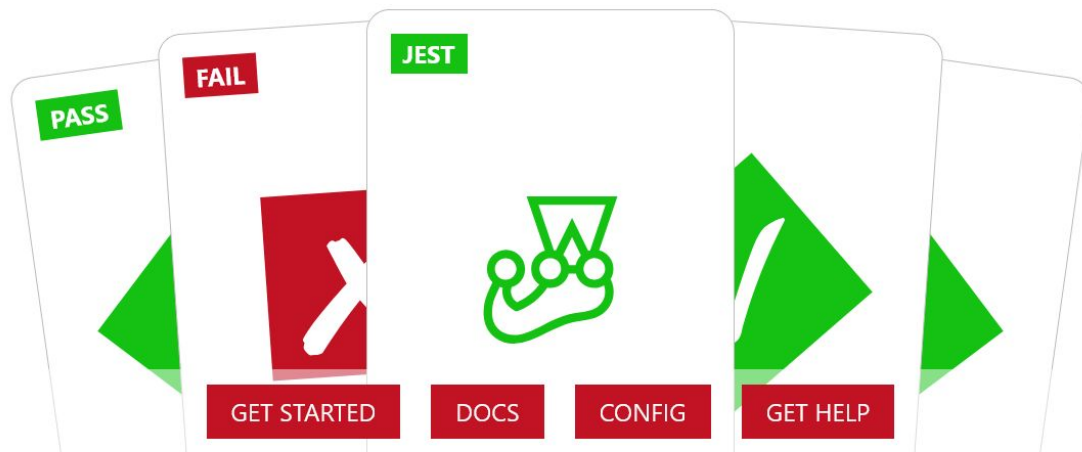
CTRL K



Follow @jestjs\_



Star



Jest is a delightful JavaScript Testing Framework  
with a focus on simplicity.



# Configuración de Jest

---

```
yarn add jest --dev
```

```
{  
  "scripts": {  
    "test": "jest -verbose"  
  }  
}
```

Todo archivo que tenga el formato **filename.test.js** será considerado como un **archivo de tests**

# Uso básico de Jest

---

Se compara el **resultado de una acción (*received*)** con lo que se **espera al realizar esa acción (*expected*)**

```
expect(received).toBe(expected);
```



Función de Jest



Matcher

# Uso básico de Jest

---

Se compara el **resultado de una acción (*received*)** con lo que se **espera al realizar esa acción (*expected*)**

```
test('adds 2 + 3 to equal 5', () => {  
  expect(add(2, 3)).toBe(5);  
});
```

# Matchers

---

Permiten comparar *received* con *expected* de diferentes maneras

([Using Matchers · Jest](#) y [Expect · Jest](#))

## Igualdad

- `toBe`
- `toEqual`

## *Truthiness*

- `toBeNull`
- `toBeUndefined`
- `toBeTruthy`
- `toBeFalsy`

## Números

- `toBeGreaterThan`
- `toBeGreaterThanOrEqual`
- `toBeLessThan`
- `toBeLessThanOrEqual`

## Strings

- `toMatch`

## Arrays

- `toContain`
- `toHaveLength`

## Objetos

- `toHaveProperty`

# Funciones asíncronas

---

```
async function asyncAdd(a, b) {  
  return a + b;  
}
```

En caso de querer testear funciones asíncronas, podemos utilizar una aproximación basada en promesas o basada en `async/await`

```
test('(Promise) adds 2 + 3 to equal 5', () => {  
  return asyncAdd(2, 3).then(result => {  
    expect(result).toBe(5);  
  });  
});  
  
test('(Promise resolves) adds 2 + 3 to equal 5', () => {  
  return expect(asyncAdd(2, 3)).resolves.toBe(5);  
});
```

```
test('(async/await) adds 2 + 3 to equal 5', async () => {  
  const result = await asyncAdd(2, 3);  
  expect(result).toBe(5);  
});
```

# Mocks

---

Podemos reemplazar la implementación real de una función por lo que queramos

```
const myFn = jest.fn();
const myImplementedFn = jest.fn(() => 'return value');

myFn();
myFn();
myImplementedFn();

console.log(myFn.mock.calls);
console.log(myImplementedFn.mock.results);
```

# Ejemplo: Jest + GitHub Actions

---

1. En un repositorio local, crear proyecto con yarn y agregar Jest:  
`yarn init -y`  
`yarn add jest --dev`
2. **Crear código en `sum.js`**
3. Crear test en `sum.test.js`
4. Agregar comando jest a `package.json`
5. Crear `.github/workflows/jest.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio

```
function sum(a, b) {  
  return a + b;  
}  
  
module.exports = sum;
```

# Ejemplo: Jest + GitHub Actions

---

1. En un repositorio local, crear proyecto con yarn y agregar Jest:  
yarn init -y  
yarn add jest --dev
2. Crear código en `sum.js`
3. **Crear test en `sum.test.js`**
4. Agregar comando jest a `package.json`
5. Crear `.github/workflows/jest.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio

```
const sum = require('./sum');

test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});
```



# Ejemplo: Jest + GitHub Actions

---

1. En un repositorio local, crear proyecto con yarn y agregar Jest:  
`yarn init -y`  
`yarn add jest --dev`
2. Crear código en `sum.js`
3. Crear test en `sum.test.js`
4. **Agregar comando jest a `package.json`**
5. Crear `.github/workflows/jest.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio

```
{  
  // ...  
  "scripts": {  
    "test": "jest"  
  },  
  // ...  
}
```

# Ejemplo: Jest + GitHub Actions

---

1. En un repositorio local, crear proyecto con yarn y agregar Jest:  
`yarn init -y`  
`yarn add jest --dev`
2. Crear código en `sum.js`
3. Crear test en `sum.test.js`
4. Agregar comando jest a `package.json`
5. Crear `.github/workflows/jest.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio

```
name: Tests
run-name: Ejecutando Jest ✨
on: [push]

jobs:
  Ejecutando-Jest:
    runs-on: ubuntu-latest
    timeout-minutes: 5 // Siempre usar
    steps:
      - uses: actions/checkout@v4
      - name: Install modules
        run: yarn
      - name: Run Jest
        run: yarn test
```

# Agenda

---

Testing

Tests de *back-end*

✨ Tests de *front-end*

# Tests de *front-end*

---

En este tipo de tests se valida que los elementos de la interfaz gráfica (GUI) funcionen como se espera. Por ejemplo, que un formulario acepta los caracteres correctos, que se muestran los mensajes de error, que la página cargue lo suficientemente rápido, etc.

¿Por qué es útil?:

- Permite identificar problemas de *performance*
- Verifica funcionalidad a través de distintos navegadores y dispositivos
- Verifica la integración con servicios externos

# Más tipos de test: regresión visual

En un **test de regresión** (*regression testing*) se verifica que los cambios en un sistema no intervengan con otras features existentes (que no se produzcan cambios no intencionados)

En el caso de los **tests de regresión visual** (*visual regression testing*) se aplica la misma lógica pero al aspecto visual, es decir, se verifica que los cambios en el código no afecten la interfaz visual más allá de lo esperado

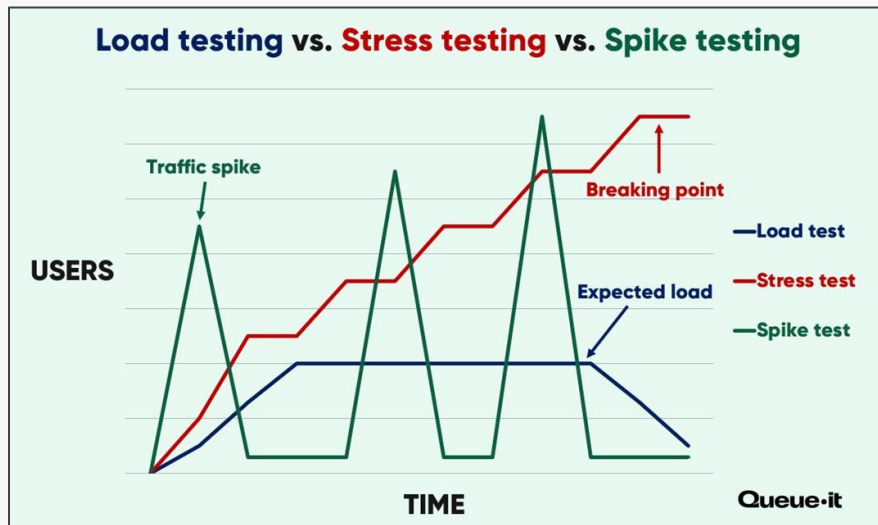


# Más tipos de test: performance

En un **test de performance** (*performance testing*) se mide el tiempo de respuesta de la aplicación ante ciertas condiciones

Simula una carga de usuarios e interacciones para identificar cuellos de botella que podrían estar afectando la experiencia de usuario

- [JMeter](#)
- [BlazeMeter](#)

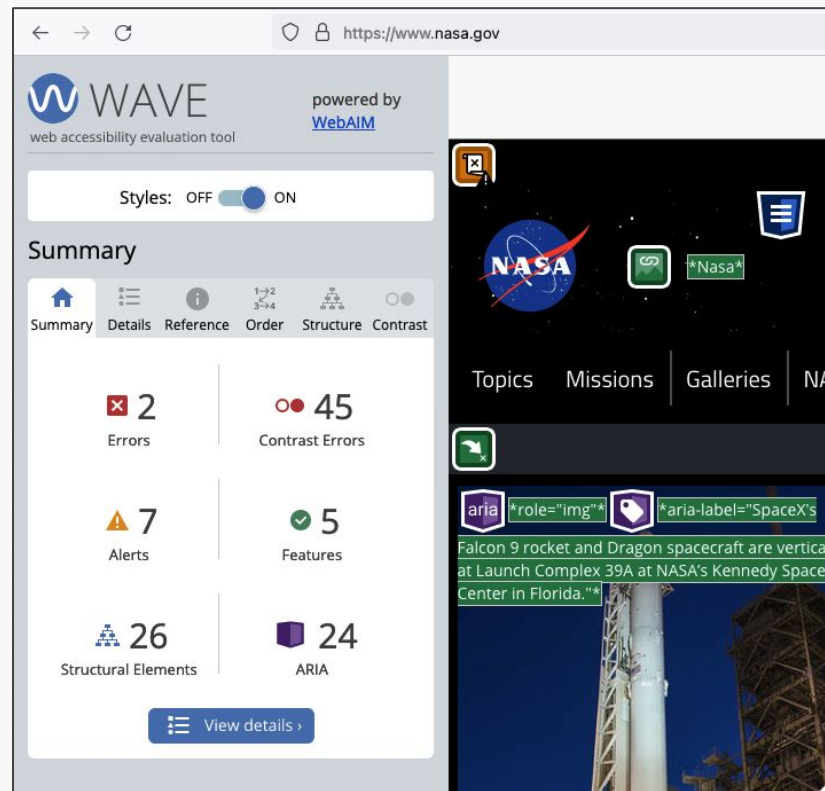


# Más tipos de test: accesibilidad

En un **test de accesibilidad** (*accessibility testing*) se verifica que la mayor cantidad de usuarios pueda acceder e interactuar con la aplicación

Para este propósito, se deben considerar aspectos como dificultades visual u otras necesidades similares

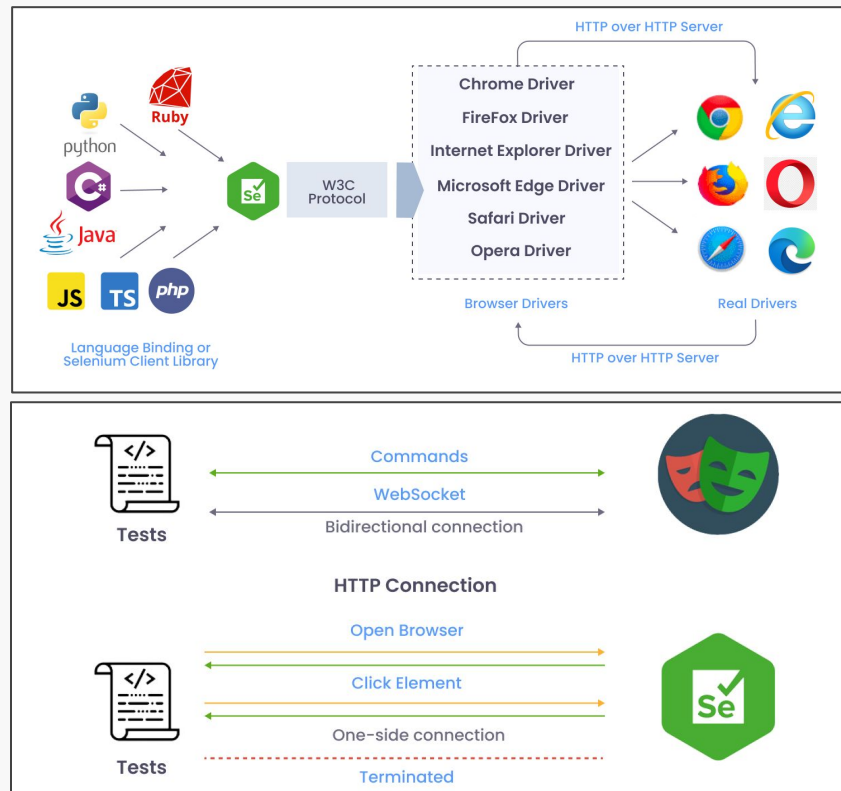
- [WAVE Web Accessibility Evaluation Tools](#)



# Más tipos de test: end-to-end (E2E)

En un **test end-to-end** (E2E) se verifica que la aplicación funcione sin problemas desde el comienzo hasta el final, siguiendo un escenario basado en el mundo real emulando un usuario real por medio de la interfaz gráfica de la aplicación

- [Selenium](#)
- [Playwright](#) (\*)





# Ejemplo: Emulando un usuario con Selenium

---

```
test('Successful login', async () => {  
    // Send credentials  
    await driver.findElement(webdriver.By.id('username')).sendKeys(TEST_USER);  
    await driver  
        .findElement(webdriver.By.xpath('//*[@type="password"]'))  
        .sendKeys(TEST_PWD);  
  
    // Click and wait for answer  
    await driver.findElement(webdriver.By.css('button[type=submit]')).click();  
    output = await driver.wait(  
        webdriver.until.elementLocated(webdriver.By.id('logged-in')),  
        1000  
    );  
    outputVal = await output.getAttribute('innerHTML');  
  
    // Matching answer  
    expect(outputVal).toEqual('You are now logged in!');  
});
```



# Testing Library

Simple and complete testing utilities that encourage good testing practices



[Get Started](#)

*The more your tests resemble the way your software is used,  
the more confidence they can give you.*

# Example | Testing Library:

## Ejemplo de React Testing Library



Testing Library

Docs

Examples

Help

Blog



Search

CTRL K

FAQ

Core API

Frameworks

DOM Testing Library

React Testing Library

Introduction

Example

Setup

API

Migrate from Enzyme

FAQ

Cheatsheet

Vue Testing Library

Angular Testing Library

Svelte Testing Library



Frameworks

React Testing Library

Example

# Example

## Quickstart

This is a minimal setup to get you started. If you want to see a description of what each line does, scroll down to the [annotated version](#). Scroll down to [Full Example](#) to see a more advanced test setup.

```
import {render, screen} from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import '@testing-library/jest-dom'
import Fetch from './fetch'
```

```
test('loads and displays greeting', async () => {
  // ARRANGE
  render(<Fetch url="/greeting" />)

  // ACT
```

Quickstart

Full Example

Step-By-Step

Imports

Mock

Arrange

Act

Assert

System Under Test

# Ejemplo: Tests sobre React con RTL

---

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import '@testing-library/jest-dom';
import Fetch from './fetch';

test('loads and displays greeting', async () => {
  // ARRANGE
  render(<Fetch url='/greeting' />);

  // ACT
  await userEvent.click(screen.getByText('Load Greeting'));
  await screen.findByRole('heading');

  // ASSERT
  expect(screen.getByRole('heading')).toHaveTextContent('hello there');
  expect(screen.getByRole('button')).toBeDisabled();
});
```

# Ejemplo: Tests sobre React con RTL

---

```
import { cleanup, fireEvent, render } from '@testing-library/react';
import CheckboxWithLabel from '../CheckboxWithLabel';

// Note: running cleanup afterEach is done automatically for you in
// @testing-library/react@9.0.0 or higher
// unmount and cleanup DOM after the test is finished.
afterEach(cleanup);

it('CheckboxWithLabel changes the text after click', () => {
  const { queryByText, getByLabelText } = render(
    <CheckboxWithLabel labelOn='On' labelOff='Off' />
  );

  expect(queryByText(/off/i)).toBeTruthy();

  fireEvent.click(getByLabelText(/off/i));

  expect(queryByText(/on/i)).toBeTruthy();
});
```

# Agenda

---

Testing

Tests de *back-end*

Tests de *front-end*



**¿Qué se viene pronto?**

# Próximas fechas importantes: clases y ayudantías

|                                          | Clases y ayudantías |                                    |                               |                             |                                          |        |         |
|------------------------------------------|---------------------|------------------------------------|-------------------------------|-----------------------------|------------------------------------------|--------|---------|
|                                          | Lunes               | Martes                             | Miércoles                     | Jueves                      | Viernes                                  | Sábado | Domingo |
| <b>Semana 12</b><br><b>(19/5 - 25/5)</b> |                     | Charla GraphQL<br>(Daniela Flores) | Día de las Glorias<br>Navales | Almacenes de<br>información |                                          |        |         |
| <b>Semana 13</b><br><b>(26/5 - 01/6)</b> |                     | Tecnologías<br>adicionales         |                               | HOY                         | Sala de ayuda: E1                        |        |         |
| <b>Semana 14</b><br><b>(02/6 - 08/6)</b> |                     | Seguridad web                      |                               | Protocolos<br>seguros       |                                          |        |         |
| <b>Semana 15</b><br><b>(09/6 - 15/6)</b> |                     | Privacidad                         |                               | Ética en CS                 |                                          |        |         |
| <b>Semana 16</b><br><b>(16/6 - 22/6)</b> |                     | Aplicaciones en el<br>mundo real   |                               | Charla TBD                  | Día Nacional de los<br>Pueblos Indígenas |        |         |

# Próximas fechas importantes: evaluaciones

|                            | Evaluaciones |        |                            |        |                                       |           |        |         |
|----------------------------|--------------|--------|----------------------------|--------|---------------------------------------|-----------|--------|---------|
|                            | Lunes        | Martes | Miércoles                  | Jueves | Viernes                               |           | Sábado | Domingo |
| Semana 12<br>(19/5 - 25/5) | Control 3    |        | Día de las Glorias Navales |        |                                       |           |        |         |
| Semana 13<br>(26/5 - 01/6) |              |        |                            | HOY    | Fin E1                                | Inicio E2 |        |         |
| Semana 14<br>(02/6 - 08/6) | Control 4    |        |                            |        |                                       |           |        |         |
| Semana 15<br>(09/6 - 15/6) |              |        |                            |        | Fin E2                                | Inicio E3 |        |         |
| Semana 16<br>(16/6 - 22/6) | Control 5    |        |                            |        | Día Nacional de los Pueblos Indígenas |           |        |         |



Clase 18

# Testing

---

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra  
aaossa@ing.puc.cl